

ADOPT-05: Optimization Roadmap

Series: ADOPT — Observability Adoption & Maturity | **Notebook:** 5 of 6 | **Created:** March 2026 | **Last Updated:** 07/24/2026

Overview

An optimization roadmap translates maturity assessment results and success metrics into a prioritized plan of action. This notebook covers monthly milestone planning, the distinction between quick wins and strategic investments, cost optimization opportunities (bucket management, sampling, retention tuning), automation candidates, and methods for measuring the return on observability investment. The roadmap framework is designed to be adapted to any organization's size and pace.

OneAgent Attribute Enrichment (1.331+): OneAgent can enrich all telemetry (metrics, spans, logs, events) with primary fields (`dt.security_context` , `dt.cost.costcenter`) and primary tags (`primary_tags.environment` , `primary_tags.team`) at the source. More efficient than auto-tags — feeds directly into OpenPipeline routing, bucket assignment, and Grail permissions. Configure via `oneagentctl --set-host-tag` or `--set-host-tag` at install time. See [docs](#).

Dynatrace Version Support Policy

Component	Standard Support	Enterprise Support
OneAgent	9 months	12 months
ActiveGate	9 months	12 months
Dynatrace Operator	Independent release cycle — check release notes	

Technology Support Tiers

Tier	Meaning
Full Support	All monitoring features maintained, bugs fixed, enhancements delivered
Early Adopter	Newly introduced — functional but may lack full feature parity; feedback welcome
End of Life (EOL)	No updates, fixes, or enhancements; monitoring may still work but issues won't be addressed

Third-Party Technology EOL Policy

Dynatrace continues to support monitoring a third-party technology for **6 months beyond the vendor's end-of-life date**. End-of-support announcements are published 6 months in advance.

Reference: [Technology Support Model](#) | [End-of-Support Announcements](#) | [Support Policy](#)

Table of Contents

- [1. Roadmap Planning Principles](#)
 - [2. Quick Wins — Month 1](#)
 - [3. Foundation Building — Months 2-3](#)
 - [4. Strategic Investments — Months 4-6](#)
 - [5. Cost Optimization Opportunities](#)
 - [6. Automation Candidates](#)
 - [7. Measuring ROI of Observability](#)
 - [8. Summary and Series Conclusion](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	<code>storage:logs:read</code> , <code>storage:metrics:read</code> , <code>storage:entities:read</code> , <code>storage:buckets:read</code>
Context	Completed ADOPT-01 through ADOPT-04
Audience	Platform team leads, engineering directors, VP of Engineering, CTO

1. Roadmap Planning Principles

An effective optimization roadmap follows these principles:

Principle 1: Start with Impact, Not Complexity

Prioritize actions by their impact on reliability and team productivity, not by their technical sophistication. A simple alerting cleanup can reduce MTTR more than a complex automation workflow.

Principle 2: Measure Before and After

Every initiative on the roadmap should have a measurable outcome tied to the success metrics defined in ADOPT-03 (MTTR, MTTD, noise ratio, etc.).

Principle 3: Quick Wins Build Momentum

Early, visible wins build organizational trust in the observability investment. Prioritize 2-3 quick wins in the first month.

Principle 4: Budget for Learning

Every initiative has a learning component. Allocate time for team enablement (ADOPT-04) alongside technical work.

Roadmap Structure

Timeframe	Focus	Effort Level
Month 1	Quick wins — immediate impact	Low effort, high visibility
Months 2-3	Foundation — platform hygiene	Medium effort, foundational
Months 4-6	Strategic — transformative initiatives	High effort, long-term value

2. Quick Wins — Month 1

Quick wins require minimal effort but deliver visible results. They establish credibility for the broader optimization program.

2.1 Alert Noise Cleanup

Goal: Reduce alert noise ratio by 50%.

Actions:

- Identify the top 5 noisiest alert sources
- Configure maintenance windows for known noisy periods
- Adjust Dynatrace Intelligence sensitivity for over-alerting entity types
- Document alerting ownership per service

Use this query to identify the noisiest problem sources:

```
// Top 10 most frequent problem types in the last 30 days
fetch dt.davis.problems, from:-30d
| summarize
    total = count(),
    frequent = countIf(dt.davis.is_frequent_event == true),
    by:{event.name}
| fieldsAdd noise_pct = round(toDouble(frequent) / toDouble(total) * 100,
decimals: 1)
| sort total desc
| limit 10
```

2.2 Agent Version Standardization

Goal: Reduce OneAgent version sprawl to 2 or fewer major versions.

Actions:

- Identify outdated agent versions
- Enable auto-update for non-production hosts
- Schedule coordinated updates for production

```
// Agent version distribution – identify outdated versions
fetch dt.entity.host
| summarize host_count = count(), by:{agentVersion}
| sort host_count desc

// No Smartscape equivalent: agentVersion / installerVersion (OneAgent
version) is not a
// Smartscape node field. Keep the classic query above for agent-version
distribution.
```

2.3 Dashboard Consolidation

Goal: Replace ad-hoc dashboards with standardized templates.

Actions:

- Audit existing dashboards for overlap and staleness
- Create 3 standard templates: Infrastructure Health, Application Health, Business KPIs
- Retire unused dashboards

3. Foundation Building — Months 2-3

Foundation work addresses platform hygiene and establishes the infrastructure for long-term success.

3.1 Complete Agent Deployment

Goal: Achieve > 95% host monitoring coverage.

Actions:

- Compare monitored hosts against CMDB inventory
- Deploy OneAgent to all unmonitored hosts
- Establish automated agent deployment for new hosts (cloud-init, Ansible, etc.)

3.2 Grail Bucket Organization

Goal: Implement a bucket strategy aligned with data ownership and retention requirements.

Actions:

- Audit current bucket usage and data distribution
- Design a bucket naming convention (e.g., `<team>_<datatype>_<env>`)
- Configure OpenPipeline routing to direct data to appropriate buckets
- Set retention policies per bucket based on compliance and cost requirements

```
// Analyze log volume by source to inform bucket strategy
fetch logs, from:-24h
| summarize record_count = count(), by:{log.source}
| sort record_count desc
| limit 20
```

3.3 SLO Definition

Goal: Define SLOs for the top 10 business-critical services.

Actions:

- Identify the 10 services with the highest business impact
- Define availability and latency SLOs for each
- Configure burn-rate alerting
- Create an SLO dashboard for leadership visibility

3.4 IAM Policy Implementation

Goal: Move from shared admin accounts to role-based access.

Actions:

- Define IAM groups per team (see IAM series for detailed guidance)
- Implement least-privilege policies
- Configure data-scoped access (segments, management zones)

4. Strategic Investments — Months 4-6

Strategic investments are higher-effort initiatives that deliver transformative value over time.

4.1 Workflow Automation

Goal: Automate response to the top 5 most frequent problem types.

Actions:

- Identify the 5 most frequent, well-understood problem types from ADOPT-03 data
- Design Dynatrace Workflows for automated triage and notification
- Progress to automated remediation for problems with known runbooks
- Measure MTTR improvement for automated vs manual resolution

4.2 Configuration-as-Code (Monaco)

Goal: Manage Dynatrace configuration through version-controlled code.

Actions:

- Export current configuration to Monaco format
- Establish a Git repository for Dynatrace configuration
- Implement CI/CD pipeline for configuration deployment
- Enable drift detection to catch manual changes

4.3 OpenTelemetry Integration

Goal: Extend observability to applications not covered by OneAgent.

Actions:

- Identify services that need custom instrumentation
- Implement OpenTelemetry SDKs for critical code paths
- Route OTel data through Dynatrace's OTLP endpoint
- Validate trace context propagation end-to-end

4.4 Business Event Correlation

Goal: Connect technical metrics to business outcomes.

Actions:

- Identify 3-5 key business transactions (checkout, login, search, etc.)
- Instrument business events for each transaction
- Build dashboards correlating technical health with business KPIs
- Establish business-context alerting (e.g., revenue impact of slowdowns)

5. Cost Optimization Opportunities

Cost optimization is not about reducing observability — it is about ensuring every byte of data delivers value. The following queries help identify opportunities.

5.1 High-Volume Log Sources

Identify the sources generating the most log data. High-volume sources are candidates for filtering, sampling, or reduced retention.

```
// Top 15 log sources by volume in the last 7 days
fetch logs, from:-7d
| summarize record_count = count(), by:{log.source}
| sort record_count desc
| limit 15
```

5.2 Debug and Verbose Log Analysis

Debug-level logs are often the largest contributor to ingestion volume but rarely needed in production. This query quantifies the opportunity.

```
// Log volume breakdown by severity level over the last 24 hours
fetch logs, from:-24h
| summarize record_count = count(), by:{loglevel}
| sort record_count desc
```

Cost optimization actions for high-volume logs:

- **Filter at source:** Configure OpenPipeline to drop DEBUG/TRACE logs in production
- **Reduce retention:** Route verbose logs to buckets with shorter retention (7-14 days)
- **Sample:** Use sampling for high-cardinality, low-value data
- **Aggregate:** Replace raw log storage with metric extraction for patterns you only need to count

5.3 Entity Monitoring Mode Review

Not every host requires full-stack monitoring. Infrastructure-only mode costs fewer host units and may be appropriate for non-application hosts.

```
// Host count by monitoring mode – identify over-provisioned monitoring
fetch dt.entity.host
| summarize host_count = count(), by:{monitoringMode}
| sort host_count desc

// No Smartscape equivalent: monitoringMode is a classic host property, not a
Smartscape node
// field. Keep the classic query above for monitoring-mode breakdowns.
```

5.4 Span Volume Analysis

Distributed tracing can generate significant data volume. Identify services producing the most spans to evaluate whether head-based or tail-based sampling is appropriate.

```
// Top 10 services by span volume in the last 24 hours
fetch spans, from:-24h
| summarize span_count = count(), by:{dt.entity.service}
| sort span_count desc
| limit 10
```

Cost Optimization Decision Matrix

Data Type	Low Value	Medium Value	High Value
Logs	Drop DEBUG/TRACE	7-day retention	35-day retention
Spans	Sample at 10:1	Sample at 2:1	Full retention
Metrics	Reduce resolution	Standard resolution	Full resolution
Events	Filter noise	Standard retention	Full retention

6. Automation Candidates

Automation reduces toil and improves consistency. Prioritize automating tasks that are repetitive, well-defined, and currently manual.

Automation Priority Matrix

Task	Frequency	Manual Effort	Automation Tool	Priority
Alert triage and routing	Daily	30 min/day	Dynatrace Workflows	High
Agent deployment on new hosts	Weekly	1 hour/week	Cloud-init / Ansible	High
Dashboard creation for new services	Monthly	2 hours/service	Monaco templates	Medium
Configuration drift detection	Weekly	1 hour/week	Monaco CI/CD	Medium
Report generation for leadership	Weekly	2 hours/week	Scheduled DQL notebooks	Medium
Incident postmortem data collection	Per incident	1 hour/incident	Workflow + DQL	Low (high value)
Capacity forecasting	Monthly	4 hours/month	Dynatrace Intelligence forecasting	Low (high value)

Quick-Start Automation: Noisy Alert Suppression

The simplest and highest-impact automation is routing noisy alerts. Use this query to identify candidates:

```
// Problems that auto-resolve within 5 minutes – candidates for suppression
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| fieldsAdd duration_minutes = resolved_problem_duration / 1m
| filter duration_minutes < 5
| summarize auto_resolve_count = count(), by:{event.name}
| sort auto_resolve_count desc
| limit 10
```

Interpretation: Problems that consistently resolve within 5 minutes are strong candidates for:

- Delayed notification (wait 5 minutes before alerting)
- Automated acknowledgment
- Workflow-based triage that only escalates if the problem persists

7. Measuring ROI of Observability

Leadership will eventually ask: **"What is the return on our observability investment?"**

This section provides a framework for answering that question.

7.1 Cost Avoidance

Calculate the cost of incidents prevented or shortened by improved observability.

Metric	Formula	Example
Incident cost reduction	(Old MTTR - New MTTR) x Avg incidents/month x Cost per hour of downtime	(2h - 0.5h) x 20 x \$10,000 = \$300,000/month
Alert fatigue reduction	(Old noise hours - New noise hours) x Avg engineer hourly rate	(20h - 5h) x \$75 = \$1,125/week
Prevented outages	Proactive detections x Estimated outage cost	5 x \$50,000 = \$250,000/quarter

7.2 Productivity Gains

Metric	Formula	Example
Self-service investigation	Escalation tickets eliminated x Avg ticket resolution time x Rate	100 tickets x 2h x \$75 = \$15,000/month
Onboarding acceleration	Weeks saved per new hire x New hires per year x Weekly rate	2 weeks x 20 hires x \$3,750 = \$150,000/year

7.3 Business Value Metrics

Metric	How to Measure
Revenue protected	SLO compliance rate x Revenue at risk
Customer satisfaction	Correlation between uptime and NPS/CSAT
Deployment velocity	Change failure rate improvement enables faster releases

7.4 Building the ROI Dashboard

Create a monthly ROI report combining:

1. MTTR/MTTD trends (from ADOPT-03)
2. Problem count trends (declining = improving)
3. Data volume and cost metrics (from this notebook)
4. Team enablement progress (from ADOPT-04)
5. Estimated cost avoidance (calculated from the formulas above)

8. Summary and Series Conclusion

Key Takeaways

- Start with quick wins in Month 1 to build organizational momentum
- Cost optimization is about value alignment, not cost cutting — every byte should earn its storage
- Automation candidates should be prioritized by frequency and manual effort
- ROI of observability can be quantified through cost avoidance, productivity gains, and business value

- The roadmap is a living document — review and adjust monthly

ADOPT Series Summary

Notebook	Focus	Key Outcome
ADOPT-01	Maturity Model	Understand where you are today
ADOPT-02	Platform Health	Assess deployment completeness
ADOPT-03	Success Metrics	Define MTTR, MTTD, and baselines
ADOPT-04	Team Enablement	Build organizational capability
ADOPT-05	Optimization Roadmap	Plan the path forward

What Comes Next

With the ADOPT series complete, your organization has:

- A clear understanding of its current maturity level
- A platform health baseline
- Defined success metrics with targets
- A team enablement plan
- A prioritized optimization roadmap

The next step is execution. Use the other notebook series in this repository (ONBRD, K8S, IAM, AUTOM, WFLOW, etc.) as practical guides for implementing each roadmap initiative.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.